অধ্যায় ৫

Functions এবং Scope

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Function Declaration, Function Expression এবং Arrow Function লিখতে ও তাদের পার্থক্য বলতে।
- Parameter ও Argument-এর পার্থক্য ব্যাখ্যা করতে।
- `return` দিয়ে মান ফেরত দিতে এবং `return` ও `console.log()`-এর পার্থক্য বুঝতে।
- Default Parameter ব্যবহার করতে।
- Block Scope, Function Scope এবং Global Scope চিনতে।
- Lexical Scope কীভাবে ঠিক করে কোন Variable কোথা থেকে পাওয়া যাবে, তা বুঝতে।
- Hoisting-এর মৌলিক আচরণ ব্যাখ্যা করতে।
- বড় কাজকে ছোট, পুনর্ব্যবহারযোগ্য Function-এ ভাগ করতে।
- একটি সম্পূর্ণ **Invoice Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৪ সম্পন্ন করা: `for` Loop এবং Array-র প্রাথমিক ব্যবহার।
- অধ্যায় ২-এর `let`, `const` ও Block; অধ্যায় ৩-এর শর্ত ও Ternary।
- অধ্যায় ১ থেকে এ পর্যন্ত আপনি Function ব্যবহার করে আসছেন; এই অধ্যায়ে সেটির পূর্ণ ব্যাখ্যা।
- `chapter-05` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

গত চারটি অধ্যায়ে আপনি `toMark`, `calculateOrder`, `analyzeCart`-এর মতো Function লিখেছেন, কারণ Function ছাড়া অর্থপূর্ণ Program লেখা প্রায় অসম্ভব। কিন্তু সেগুলো ছিল একটি নির্দিষ্ট ছাঁচে লেখা। Function আসলে কীভাবে কাজ করে, কত রকমভাবে লেখা যায় এবং Function-এর ভেতরের Variable বাইরে থেকে কেন দেখা যায় না—এসব এখনো ব্যাখ্যা করা হয়নি।

Function হলো Program গঠনের মূল একক। একটি বড় সমস্যাকে ছোট ছোট নামযুক্ত কাজে ভাগ করা, প্রতিটিকে আলাদাভাবে পরীক্ষা করা এবং একই কাজ বহু জায়গায় ব্যবহার করা—পেশাদার Code-এর এই তিনটি বৈশিষ্ট্যই Function থেকে আসে।

এর সঙ্গে জড়িত **Scope**: কোন নাম Program-এর কোন অংশ থেকে ব্যবহার করা যায়, তার নিয়ম। Scope না বুঝলে "Variable-টি তো আছে, তবু খুঁজে পাচ্ছে না কেন" ধরনের Error বারবার ফিরে আসে।

৫.১ Function কেন প্রয়োজন

একটি দোকানের Program-এ তিন জায়গায় পণ্যের মোট দাম হিসাব করতে হয়: Cart-এ, Invoice-এ এবং Report-এ। তিন জায়গায় একই হিসাব আলাদাভাবে লিখলে নিয়ম বদলানোর সময় তিন জায়গাতেই বদলাতে হবে, আর একটি বাদ পড়লেই Bug।

why-functions.js · JavaScript — চলবে: Browser Console ও Node.js

```
function calculateLineTotal(price, quantity) {
  return price * quantity;
}

console.log(calculateLineTotal(120, 3));
console.log(calculateLineTotal(15, 10));
```

Output

```
360
150
```

হিসাবটি একবার লেখা হয়েছে, একটি নাম দেওয়া হয়েছে, এবং দুবার ব্যবহার করা হয়েছে। Function তিনটি সুবিধা দেয়:

- **পুনর্ব্যবহার।** একবার লিখে যতবার প্রয়োজন ব্যবহার।
- **নাম।** `price * quantity`-এর চেয়ে `calculateLineTotal` বেশি স্পষ্টভাবে বলে কী করা হচ্ছে।
- **এক জায়গায় পরিবর্তন।** নিয়ম বদলালে শুধু Function-টি বদলাতে হয়।

৫.২ Function Declaration

ওপরের লেখার ধরনটিকে **Function Declaration** বলে। এর চারটি অংশ:

অংশ	উদাহরণে	কাজ
`function` শব্দ	`function`	JavaScript-কে জানায় একটি Function তৈরি হচ্ছে
নাম	`calculateLineTotal`	এই নাম দিয়ে Function-টি ব্যবহার করা হবে

অংশ	উদাহরণে	কাজ
Parameter-এর তালিকা	`(price, quantity)`	Function যে মানগুলো গ্রহণ করে
Body	`{ return price * quantity; }`	যে Code চলবে

Function তৈরি করলেই সেটি চলে না। চালাতে হলে নামের পরে বন্ধনী দিয়ে **Call** করতে হয়: `calculateLineTotal(120, 3)`। বন্ধনী ছাড়া শুধু নাম লিখলে Function-টি চলে না; নামটি তখন Function-টিকেই বোঝায়, যা অনুচ্ছেদ ৫.৬-এ কাজে লাগবে।

Function-এর নাম একটি ক্রিয়া দিয়ে শুরু করা প্রচলিত রীতি, কারণ Function কিছু একটা করে: `calculateTotal`, `formatPrice`, `isValid`, `getGrade`।

৫.৩ Parameter এবং Argument

দুটি শব্দ প্রায়ই একই অর্থে ব্যবহৃত হয়, কিন্তু এরা আলাদা।

- **Parameter** — Function তৈরির সময় বন্ধনীতে লেখা নাম। এগুলো Function-এর ভেতরের Variable।
- **Argument** — Function Call করার সময় দেওয়া আসল মান।

`function calculateLineTotal(price, quantity)`-তে `price` ও `quantity` Parameter। `calculateLineTotal(120, 3)`-তে `120` ও `3` Argument। Call-এর সময় Argument-গুলো ক্রম অনুযায়ী Parameter-এ বসে: প্রথমটি প্রথমে, দ্বিতীয়টি দ্বিতীয়তে।

সংখ্যা না মিললে

অনেক ভাষায় Argument-এর সংখ্যা ভুল হলে Error হয়। JavaScript-এ হয় না।

argument-count.js · JavaScript — চলবে: Browser Console ও Node.js

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3));
console.log(add(2));
console.log(add(2, 3, 4));
```

Output

```
5
NaN
5
```

- কম Argument দিলে বাকি Parameter-গুলো `undefined` হয়। `2 + undefined` এর ফল `NaN`।
- বেশি দিলে অতিরিক্তগুলো নীরবে উপেক্ষিত হয়।

কোনো ক্ষেত্রেই Error Message আসে না। তাই Function Call-এর সময় Argument-এর সংখ্যা ও ক্রম মিলিয়ে নেওয়া আপনার দায়িত্ব। ক্রম উল্টে গেলেও (`calculateLineTotal(3, 120)`) JavaScript কিছু বলবে না।

৫.8 Return Value

`return` দুটি কাজ করে: Function-এর ফলাফল Call-এর জায়গায় ফেরত পাঠায়, এবং Function সেখানেই শেষ করে দেয়।

নতুনদের সবচেয়ে প্রচলিত বিভ্রান্তি হলো `return` এবং `console.log()` গুলিয়ে ফেলা।

return-vs-log.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function showTotal(price, quantity) {
  console.log(price * quantity);
}

function getTotal(price, quantity) {
  return price * quantity;
}

const shown = showTotal(120, 3);
const returned = getTotal(120, 3);

console.log(shown);
console.log(returned);
console.log(getTotal(120, 3) + 50);
```

Output

```
360
undefined
360
410
```

`showTotal` পর্দায় `360` দেখায়, কিন্তু কিছু ফেরত দেয় না; তাই `shown`-এর মান `undefined`। যে Function-এ `return` নেই, অথবা `return`-এর পরে কোনো মান নেই, সেটি সব সময় `undefined` ফেরত দেয়।

`getTotal` কিছু দেখায় না, কিন্তু মানটি ফেরত দেয়। সেই মান Variable-এ রাখা যায়, অন্য হিসাবে ব্যবহার করা যায় (শেষ লাইনে `+ 50`), অথবা অন্য Function-এ পাঠানো যায়।

	`console.log()`	`return`
কার জন্য	Developer-এর চোখের জন্য	Program-এর বাকি অংশের জন্য
মানটি পরে ব্যবহার করা যায়	না	হ্যাঁ
Function থামায়	না	হ্যাঁ

> **মনে রাখুন:** যে Function হিসাব করে, সেটি ফল `return` করবে; দেখাবে না। দেখানোর কাজ আলাদা Function-এ রাখুন। তাহলে একই হিসাব পর্দায়, File-এ বা Test-এ—সব জায়গায় ব্যবহার করা যায়।

return Function থামিয়ে দেয়

`return` চলার পর Function-এর আর কোনো লাইন চলে না। অধ্যায় ৩-এর Guard Clause এই আচরণের ওপরই দাঁড়িয়ে।

early-return.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describeStock(quantity) {
  if (quantity <= 0) {
    return 'শেষ হয়ে গেছে';
  }

  if (quantity < 5) {
    return 'অল্প বাকি';
  }

  return 'মজুত আছে';
}

console.log(describeStock(0));
console.log(describeStock(3));
console.log(describeStock(40));
```

Output

```
শেষ হয়ে গেছে
অল্প বাকি
মজুত আছে
```

একটি Function একটিই মান ফেরত দিতে পারে। একাধিক তথ্য ফেরত দিতে হলে সেগুলোকে একটি Object বা Array-তে রেখে ফেরত দিতে হয়, যেমন অধ্যায় ৪-এর `analyzeCart` করেছিল।

৫.৫ Default Parameter

কোনো Argument না দিলে Parameter-টি `undefined` হয়। **Default Parameter** দিয়ে এমন ক্ষেত্রের জন্য একটি পূর্বনির্ধারিত মান ঠিক করে রাখা যায়।

default-parameter.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greet(name = 'অতিথি') {
  return `স্বাগতম, ${name}!`;
}

console.log(greet('Sara'));
console.log(greet());
console.log(greet(undefined));
console.log(greet(null));
```

Output

```
স্বাগতম, Sara!
স্বাগতম, অতিথি!
স্বাগতম, অতিথি!
স্বাগতম, null!
```

Default মান বসে শুধু তখনই, যখন Argument দেওয়া হয়নি অথবা `undefined` দেওয়া হয়েছে। `null` একটি ইচ্ছাকৃত মান (অধ্যায় ২), তাই `null` দিলে Default বসে না।

Default Parameter-গুলো তালিকার শেষে রাখুন। `function price(amount, discount = 0)` ঠিক আছে: শুধু `price(500)` লিখলেই চলে। Default-টি প্রথমে থাকলে তাকে এড়িয়ে দ্বিতীয়টি দেওয়ার সহজ উপায় থাকে না।

default-discount.js · JavaScript — চলবে: Browser Console ও Node.js

```
function finalPrice(amount, discountPercent = 0) {
  return amount - Math.round(amount * discountPercent / 100);
}

console.log(finalPrice(500));
console.log(finalPrice(500, 10));
```

Output

```
500
450
```

৫.৬ Function Expression

JavaScript-এ Function নিজেই একটি মান, ঠিক সংখ্যা বা String-এর মতো। তাই Function-কে একটি Variable-এ রাখা যায়। এভাবে লেখাকে **Function Expression** বলে।

function-expression.js · JavaScript — চলবে: Browser Console ও Node.js

```
const add = function (a, b) {
  return a + b;
};

console.log(add(2, 3));
console.log(typeof add);
```

Output

```
5
function
```

`function`-এর পরে কোনো নাম নেই; নামটি আসে Variable থেকে। এটি একটি Assignment Statement, তাই শেষে Semicolon আছে। Call করার নিয়ম একই।

Function যেহেতু একটি মান, তাকে অন্য Function-এর Argument হিসেবেও পাঠানো যায়। এই ক্ষমতা অনুচ্ছেদ ৫.১১-এ এবং অধ্যায় ৬-এ বারবার কাজে লাগবে।

৫.৭ Arrow Function

Arrow Function হলো Function Expression লেখার সংক্ষিপ্ত রূপ। `function` শব্দের বদলে Parameter-এর পরে `=>` (তীরের মতো দেখতে) বসে।

arrow-functions.js · JavaScript চলবে: Browser Console ও Node.js

```
const double = (number) => number * 2;

const isAdult = (age) => age >= 18;

const describe = (name, mark) => {
  const status = mark >= 33 ? 'উত্তীর্ণ' : 'অনুত্তীর্ণ';
  return `${name}: ${status}`;
};

const greeting = () => 'স্বাগতম!';

console.log(double(21));
console.log(isAdult(15));
console.log(describe('Sara', 80));
console.log(greeting());
```

Output

```
42
false
Sara: উত্তীর্ণ
স্বাগতম!
```

Arrow Function-এর Body দুইভাবে লেখা যায়:

- **একটি Expression, `{ }` ছাড়া।** Expression-টির মান নিজে থেকেই ফেরত যায়; `return` লিখতে হয় না। `double` এবং `isAdult` এভাবে লেখা।
- **`{ }` সহ একাধিক লাইন।** তখন `return` নিজে লিখতে হয়, যেমন `describe`-এ। `{ }` দিয়ে `return` বাদ দিলে Function `undefined` ফেরত দেবে।

Parameter না থাকলে ফাঁকা বন্ধনী `()` লিখতে হয়।

তিনটি রূপ পাশাপাশি

রূপ	উদাহরণ	উপযুক্ত ব্যবহার
Function Declaration	`function add(a, b) { return a + b; }`	Program-এর প্রধান, নামযুক্ত Function
Function Expression	`const add = function (a, b) { return a + b; };`	পুরোনো Code-এ দেখা যায়
Arrow Function	`const add = (a, b) => a + b;`	ছোট Function; অন্য Function-কে পাঠানোর Function

এই বইয়ে একটি সরল রীতি মানা হয়েছে: যে Function-এর একাধিক ধাপ আছে, সেটি Function Declaration; ছোট, এক লাইনের হিসাব Arrow Function।

> **নোট:** Arrow Function এবং সাধারণ Function-এর মধ্যে আরেকটি পার্থক্য আছে, যা `this` নামের একটি বিশেষ শব্দের সঙ্গে জড়িত। সেটি Object ও Class-এর আলোচনায়, অধ্যায় ৭ ও ১৩-তে আসবে। এ পর্যন্ত লেখা কোনো Code-এ তার প্রভাব নেই।

৫.৮ Scope: Block, Function এবং Global

Scope হলো Program-এর সেই অংশ, যেখান থেকে একটি Variable ব্যবহার করা যায়। JavaScript-এ তিন স্তরের Scope আছে।

Scope	কোথায় তৈরি Variable	কোথা থেকে পাওয়া যায়
Global	সব Function ও Block-এর বাইরে	File-এর সব জায়গা থেকে
Function	একটি Function-এর ভেতরে	শুধু সেই Function-এর ভেতর থেকে
Block	একটি `{ }`-এর ভেতরে (`if`, `for` ইত্যাদি), `let` বা `const` দিয়ে	শুধু সেই Block-এর ভেতর থেকে

scopes.js · JavaScript — চলবে: Browser Console ও Node.js

```
const shopName = 'Boi Ghar';

function printReceipt() {
  const total = 500;

  if (total > 100) {
    const note = 'বড় কেনাকাটা';
    console.log(shopName, total, note);
  }

  console.log(typeof note);
}

printReceipt();
console.log(typeof total);
```

Output

```
Boi Ghar 500 বড় কেনাকাটা
undefined
undefined
```

- `shopName` Global Scope-এ; Function ও Block-এর ভেতর থেকেও পাওয়া যায়।
- `total` Function Scope-এ; `printReceipt`-এর বাইরে এর অস্তিত্ব নেই।
- `note` Block Scope-এ; `if`-এর Block শেষ হলেই আর পাওয়া যায় না।

এখানে `typeof` ব্যবহার করা হয়েছে, কারণ যে নাম নেই তার ওপর `typeof` Error না দিয়ে `'undefined'` দেয়। সরাসরি ব্যবহার করতে গেলে Program থেমে যায়:

out-of-scope.js · JavaScript চলবে: Node.js

```
function printReceipt() {
  const total = 500;
}

printReceipt();
console.log(total);
```

Error Message

```
ReferenceError: total is not defined
```

নিয়মটি একমুখী: **ভেতর থেকে বাইরের Variable দেখা যায়, বাইরে থেকে ভেতরের নয়।**

Scope কেন উপকারী

Scope একটি সীমাবদ্ধতা মনে হলেও এটি আসলে সুরক্ষা। দুটি আলাদা Function-এ `total` নামে Variable থাকতে পারে, এবং একটি অন্যটিকে প্রভাবিত করে না। প্রতিটি Function নিজের নাম নিজে বেছে নিতে পারে, বাকি Program-এ কী নাম ব্যবহৃত হয়েছে তা না জেনেই।

উল্টো দিকে, Global Variable সব জায়গা থেকে বদলানো যায়। বড় Program-এ একটি Global মান ভুল হয়ে গেলে কোন Function সেটি বদলেছে, তা খুঁজে বের করা কঠিন। তাই Global Scope-এ শুধু স্থির মান (`const`) এবং Function রাখুন।

Parameter-ও Function Scope-এর Variable: `printReceipt(amount)` লিখলে `amount` শুধু ওই Function-এর ভেতরেই থাকে।

৫.৯ Lexical Scope

একটি Function-এর ভেতরে আরেকটি Function লেখা যায়। ভেতরের Function বাইরের Function-এর Variable দেখতে পায়। JavaScript একটি নাম খোঁজে ভেতর থেকে বাইরের দিকে: আগে নিজের Scope-এ, না পেলে তার বাইরের Scope-এ, এভাবে Global পর্যন্ত। এই ধারাবাহিক খোঁজার পথকে **Scope Chain** বলে।

scope-chain.js · JavaScript — চলবে: Browser Console ও Node.js

```
const rate = 5;

function outer() {
  const rate = 10;

  function inner() {
    return rate;
  }

  return inner();
}

console.log(outer());
console.log(rate);
```

Output

```
10
5
```

`inner` নিজের ভেতরে `rate` পায়নি, তাই এক ধাপ বাইরে `outer`-এ খুঁজেছে এবং সেখানে `10` পেয়ে থেমে গেছে। Global `rate` পর্যন্ত যাওয়ার দরকার হয়নি। ভেতরের Scope-এ একই নামের Variable থাকলে সেটি বাইরেরটিকে ঢেকে দেয়; একে **Shadowing** বলে। বাইরের `rate` অপরিবর্তিত থাকে।

কোথায় লেখা, সেটিই নির্ধারক

সবচেয়ে গুরুত্বপূর্ণ নিয়ম: একটি Function কোন Variable দেখতে পাবে, তা নির্ভর করে Function-টি Code-এর **কোথায় লেখা** তার ওপর; কোথা থেকে **Call করা হয়েছে** তার ওপর নয়। এই নিয়মের নাম **Lexical Scope**।

lexical-scope.js · JavaScript চলবে: Browser Console ও Node.js

```
const currency = 'টাকা';

function format(amount) {
  return `${amount} ${currency}`;
}

function report() {
  const currency = 'ডলার';
  return format(100);
}

console.log(report());
```

Output

```
100 টাকা
```

`format`-কে Call করা হয়েছে `report`-এর ভেতর থেকে, যেখানে `currency`-র মান `'ডলার'`। তবু ফল `টাকা`। কারণ `format` লেখা হয়েছে Global Scope-এ; তার Scope Chain-এ `report`-এর ভেতরের Variable নেই। `format` নিজের ভেতরে `currency` না পেয়ে সরাসরি Global-এ খুঁজেছে।

এর ব্যবহারিক অর্থ: একটি Function পড়েই বলা যায় সেটি কোন Variable ব্যবহার করবে; কে তাকে Call করছে তা জানার দরকার নেই।

৫.১০ Hoisting

JavaScript একটি File চালানোর আগে পুরোটা একবার পড়ে নেয় এবং কোন কোন নাম তৈরি হবে তা জেনে রাখে। এর ফলে কিছু নাম তাদের লেখার লাইনের আগেও পরিচিত থাকে। এই আচরণকে **Hoisting** বলে।

তিন ধরনের Declaration তিনভাবে আচরণ করে।

hoisting.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(square(4));

function square(number) {
  return number * number;
}

console.log(oldStyle);
var oldStyle = 'var দিয়ে তৈরি';
console.log(oldStyle);
```

Output

```
16
undefined
var দিয়ে তৈরি
```

- **Function Declaration** পুরোপুরি Hoist হয়: নাম এবং Body দুটোই। তাই `square` লেখার আগের লাইনেই Call করা গেছে।
- `var` আংশিক Hoist হয়: নামটি পরিচিত, কিন্তু মান তখনো `undefined`। Error আসে না, ভুল মান আসে—যা আরও বিপজ্জনক।
- `let` ও `const` দিয়ে তৈরি নাম (Function Expression ও Arrow Function-সহ) Declaration-এর লাইনের আগে ব্যবহার করা যায় না।

arrow-before.js · JavaScript চলবে: Node.js

```
console.log(cube(3));

const cube = (number) => number * number * number;
```

Error Message

```
ReferenceError: Cannot access 'cube' before initialization
```

Declaration	লেখার আগে ব্যবহার করলে
`function name() { }`	কাজ করে
`var name`	`undefined`
`let name`, `const name`	`ReferenceError`

Hoisting-এর ওপর নির্ভর করে Code লিখবেন না। Function Declaration-এর এই সুবিধা একটি কাজে লাগে: File-এর ওপরে মূল ধাপগুলো এবং নিচে সহায়ক Function-গুলো রাখা যায়, যাতে পাঠক আগে বড় ছবিটি দেখেন।

৫.১১ পুনর্ব্যবহারযোগ্য Logic

Function লেখা সহজ; ভালো Function লেখা একটি দক্ষতা। তিনটি নীতি বেশির ভাগ ক্ষেত্রে পথ দেখায়।

একটি Function, একটি কাজ

একটি Function যদি যাচাই করে, হিসাব করে এবং ফল দেখায়, তবে তার তিনটির যেকোনো একটি আলাদাভাবে ব্যবহার করা যায় না। কাজগুলো ভাগ করলে প্রতিটি অংশ আলাদাভাবে ব্যবহার ও পরীক্ষা করা যায়।

small-functions.js · JavaScript — চলবে: Browser Console ও Node.js

```
function percentOf(amount, percent) {
  return Math.round(amount * percent / 100);
}

function applyDiscount(amount, percent) {
  return amount - percentOf(amount, percent);
}

function addTax(amount, percent) {
  return amount + percentOf(amount, percent);
}

const discounted = applyDiscount(1000, 10);
const payable = addTax(discounted, 5);

console.log(discounted);
console.log(payable);
```

Output

```
900
945
```

`percentOf` দুটি ভিন্ন Function ব্যবহার করছে। শতাংশ হিসাবের নিয়ম বদলালে (যেমন পূর্ণ টাকায় আনার পদ্ধতি) এক জায়গায় বদলালেই দুটোই ঠিক হয়ে যাবে।

Pure Function

যে Function একই Argument-এ সব সময় একই ফল দেয় এবং বাইরের কিছু বদলায় না, তাকে **Pure Function** বলে। ওপরের তিনটিই Pure। এর বিপরীতে, বাইরের Variable বদলানো, পর্দায় দেখানো বা File লেখাকে **Side Effect** বলে।

pure-vs-impure.js · JavaScript চলবে: Browser Console ও Node.js

```
let runningTotal = 0;

function addToTotal(amount) {
  runningTotal += amount;
  return runningTotal;
}

function add(a, b) {
  return a + b;
}

console.log(addToTotal(100));
console.log(addToTotal(100));
console.log(add(100, 100));
console.log(add(100, 100));
```

Output

```
100
200
200
200
```

`addToTotal(100)` দুবার Call করে দুটি ভিন্ন ফল পাওয়া গেছে, কারণ এটি বাইরের `runningTotal`-এর ওপর নির্ভর করে এবং সেটি বদলায়। এর ফল বুঝতে হলে আগে কতবার Call হয়েছে তা জানতে হয়। `add` শুধু নিজের Argument-এর ওপর নির্ভর করে; এটি বোঝা, পরীক্ষা করা এবং পুনর্ব্যবহার করা সহজ।

Side Effect ছাড়া Program অর্থহীন—কোনো এক সময় ফল দেখাতেই হবে। লক্ষ্য হলো হিসাবের অংশ Pure রাখা এবং Side Effect-গুলোকে অল্প কয়েকটি Function-এ সীমাবদ্ধ করা।

Function-কে Argument হিসেবে পাঠানো

Function একটি মান, তাই একটি Function আরেকটি Function গ্রহণ করতে পারে। এতে "কী করতে হবে" আর "কোন নিয়মে করতে হবে" আলাদা করা যায়।

function-as-argument.js · JavaScript চলবে: Browser Console ও Node.js

```
const tenPercent = (amount) => Math.round(amount * 10 / 100);
const flatFifty = (amount) => 50;

function finalPrice(amount, discountRule) {
  return amount - discountRule(amount);
}

console.log(finalPrice(1000, tenPercent));
console.log(finalPrice(1000, flatFifty));
```

Output

```
900
950
```

`finalPrice` জানে না ছাড়ের নিয়ম কী; সে শুধু পাওয়া Function-টি Call করে। নতুন নিয়ম যোগ করতে `finalPrice` বদলাতে হবে না। লক্ষ করুন, `tenPercent` পাঠানো হয়েছে বন্ধনী ছাড়া: Function-টিকে পাঠানো হচ্ছে, তার ফলাফলকে নয়। অন্য Function-কে পাঠানো এমন Function-কে **Callback** বলে। অধ্যায় ৬-এর `map`, `filter` এবং `reduce` সম্পূর্ণভাবে এই ধারণার ওপর দাঁড়িয়ে।

৫.১২ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`return` লিখতে ভুলে যাওয়া	Function `undefined` ফেরত দেয়	হিসাবের ফল `return` করুন
`return`-এর বদলে `console.log()`	পর্দায় দেখায়, কিন্তু মান ব্যবহার করা যায় না	দেখানো আর ফেরত দেওয়া আলাদা
`{ }`-সহ Arrow Function-এ `return` নেই	`undefined`	`return` যোগ করুন, অথবা `{ }` সরান
বন্ধনী ছাড়া Call (`getTotal`)	Function-টির Code দেখায়, ফল নয়	`getTotal()`
Argument-এর ক্রম ভুল	ভুল ফল, কোনো Error নয়	Parameter-এর ক্রমের সঙ্গে মিলিয়ে নিন
কম Argument দেওয়া	`NaN` বা `undefined`	সব Argument দিন, অথবা Default Parameter ব্যবহার করুন
`null`-এ Default বসবে ধরে নেওয়া	`null` থেকে যায়	Default শুধু `undefined`-এর জন্য
Function-এর ভেতরের Variable বাইরে ব্যবহার	`ReferenceError: ... is not defined`	মানটি `return` করুন

ভুল	কী দেখা যায়	সমাধান
Arrow Function তৈরির আগে Call	`ReferenceError: Cannot access ... before initialization`	আগে তৈরি করুন
Function-এর ভেতরে Global Variable বদলানো	একই Call-এ ভিন্ন ভিন্ন ফল	মান Argument হিসেবে নিন, ফল `return` করুন

তৃতীয় ভুলটি বিশেষভাবে ধরা কঠিন, কারণ Code দেখতে ঠিকই লাগে।

arrow-no-return.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wrongDouble = (number) => {
  number * 2;
};

const rightDouble = (number) => number * 2;

console.log(wrongDouble(21));
console.log(rightDouble(21));
```

Output

```
undefined
42
```

৫.১৩ Debugging নির্দেশিকা

Function-সংক্রান্ত অধিকাংশ Bug দুটি প্রশ্নে ধরা পড়ে: Function-টি কী পেয়েছে, এবং কী ফেরত দিয়েছে?

debug-function.js · JavaScript — চলবে: Browser Console ও Node.js

```
function applyDiscount(amount, percent) {
  console.log('পেয়েছে:', amount, percent, typeof amount);

  const result = amount - Math.round(amount * percent / 100);

  console.log('ফেরত দিচ্ছে:', result);
  return result;
}

applyDiscount('1000', 10);
```

Output

```
পেয়েছে: 1000 10 string
ফেরত দিচ্ছে: 900
```

`typeof` দেখাচ্ছে `amount` একটি String হিসেবে এসেছে। এখানে ফল ঠিক এসেছে, কারণ `*` ও `-` String-কে সংখ্যায় রূপান্তর করে; কিন্তু পরে কেউ `+` ব্যবহার করলে জোড়া লেগে যাবে। Function-এর শুরুতে Argument দেখা এমন লুকানো সমস্যা ধরিয়ে দেয়।

Function প্রত্যাশামতো কাজ না করলে:

১. Function-এর প্রথম লাইনে প্রতিটি Parameter-এর মান ও `typeof` দেখান।
২. `return`-এর ঠিক আগে ফলটি দেখান।
৩. ফল `undefined` হলে দেখুন `return` আছে কি না, এবং সব পথে আছে কি না।
৪. Function-টি আদৌ Call হচ্ছে কি না দেখুন; নাম লিখে বন্ধনী দিতে ভুলে যাননি তো?
৫. `ReferenceError` এলে নামটির বানান এবং Scope দেখুন: Variable-টি কোন `{ }`-এর ভেতরে তৈরি?
৬. Error Message-এর Stack Trace পড়ুন (অধ্যায় ১): কোন Function থেকে কোনটি Call হয়েছে।
৭. Function-টিকে একা, নির্দিষ্ট Argument দিয়ে Call করে দেখুন। Pure Function হলে এটি সহজ; সেটিই Pure Function-এর বড় সুবিধা।

৫.১৪ পেশাদার অভ্যাস

- **একটি Function, একটি কাজ।** নামে "এবং" বসাতে হলে (`validateAndSave`) সেটি সম্ভবত দুটি Function।
- **নাম ক্রিয়া দিয়ে শুরু করুন;** সত্য-মিথ্যা ফেরত দেওয়া Function-এর নাম `is` বা `has` দিয়ে: `isValid`, `hasStock`।
- **Function ছোট রাখুন।** এক পর্দায় না আঁটলে ভাগ করার কথা ভাবুন।
- **Parameter কম রাখুন।** তিন-চারটির বেশি হলে ক্রম মনে রাখা কঠিন; তখন একটি Object নিন।
- **হিসাব আর দেখানো আলাদা রাখুন।**
- **Argument হিসেবে পাওয়া Object বদলাবেন না** (অধ্যায় ২)। নতুন মান তৈরি করে ফেরত দিন।
- **Global Variable-এর ওপর নির্ভর করবেন না।** যা দরকার Argument হিসেবে নিন।
- **সব পথে একই ধরনের মান ফেরত দিন।** কখনো সংখ্যা, কখনো String ফেরত দিলে Call-কারীকে প্রতিবার যাচাই করতে হয়।
- **Function-এর ওপরে এক লাইনের Comment** লিখুন: এটি কী নেয় এবং কী ফেরত দেয়।

৫.১৫ Project: Invoice Calculator

একটি দোকানের Invoice তৈরির Program লিখবেন: প্রতিটি পণ্যের লাইন, উপমোট, ছাড়, কর এবং সর্বমোট। এখানে মূল লক্ষ্য হিসাব নয়, বরং কাজটিকে ছোট ছোট Function-এ ভাগ করা।

Project-এর চাহিদা

- প্রতিটি পণ্যের নাম, একটির দাম এবং পরিমাণ থাকবে।
- **টাকার অঙ্ক পয়সায়, পূর্ণসংখ্যায় রাখা হবে** (১২০.৫০ টাকা = ১২০৫০ পয়সা), যাতে দশমিক হিসাবের ত্রুটি না হয়। দেখানোর সময় টাকায় রূপান্তর করা হবে।
- ছাড় ও করের হার শতাংশে দেওয়া যাবে; না দিলে দুটোই শূন্য ধরা হবে।
- ছাড় হিসাব হবে উপমোটের ওপর; কর হিসাব হবে ছাড়ের পরের অঙ্কের ওপর। প্রতিটি নিকটতম পয়সায় আনা হবে।
- হিসাবের Function কিছু দেখাবে না; শুধু ফল ফেরত দেবে।

এখানে করের হারটি একটি উদাহরণ; এটি কোনো দেশের প্রকৃত কর-নিয়ম নয়।

File-এর কাঠামো

Folder-এর কাঠামো

```
invoice-calculator/
├── invoice.js
├── README.md
└── tests/
    └── invoice.test.js
```

invoice.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: তিনটি ছোট সহায়ক Function

invoice.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
'use strict';

const PAISA_PER_TAKA = 100;

// পয়সাকে টাকার লিখিত রূপে আনে: 12050 থেকে '120.50 টাকা'।
const formatTaka = (paisa) => `${(paisa / PAISA_PER_TAKA).toFixed(2)} টাকা`;

// একটি পণ্যের লাইনের মোট: দাম x পরিমাণ (পয়সায়)।
const lineTotal = (item) => item.pricePaisa * item.quantity;

// একটি অঙ্কের নির্দিষ্ট শতাংশ, নিকটতম পয়সায়।
function percentOf(amount, percent) {
  return Math.round(amount * percent / 100);
}

// ...
```

তিনটিই Pure Function, এবং প্রতিটি একটি কাজ করে। প্রথম দুটি এক লাইনের হিসাব, তাই Arrow Function। `formatTaka`-ই একমাত্র জায়গা, যেখানে পয়সা টাকায় রূপান্তরিত হয়; বাকি সব হিসাব পূর্ণসংখ্যায় থাকে।

ধাপ ২: Invoice-এর হিসাব

invoice.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// পণ্যের তালিকা ও হার থেকে Invoice-এর চারটি অঙ্ক (পয়সায়) হিসাব করে।
function calculateInvoice(items, discountPercent = 0, taxPercent = 0) {
  let subtotal = 0;

  for (let i = 0; i < items.length; i++) {
    subtotal += lineTotal(items[i]);
  }

  const discount = percentOf(subtotal, discountPercent);
  const taxable = subtotal - discount;
  const tax = percentOf(taxable, taxPercent);

  return {
    subtotal: subtotal,
    discount: discount,
    tax: tax,
    total: taxable + tax,
  };
}

// ...
```

দুটি Default Parameter থাকায় `calculateInvoice(items)` লিখলেই ছাড় ও কর ছাড়া হিসাব পাওয়া যায়। Function-টি কিছু দেখায় না এবং `items` বদলায় না; চারটি অঙ্ক একটি Object-এ ফেরত দেয়।

ধাপ ৩: Invoice দেখানো

invoice.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// প্রতিটি লাইন এবং শেষে চারটি অঙ্ক দেখায়।
function printInvoice(items, discountPercent = 0, taxPercent = 0) {
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const label = `${item.name} x ${item.quantity}`;

    console.log(`${label}: ${formatTaka(lineTotal(item))}`);
  }

  const invoice = calculateInvoice(items, discountPercent, taxPercent);

  console.log(`উপমোট: ${formatTaka(invoice.subtotal)}`);
  console.log(`ছাড় (${discountPercent}%): ${formatTaka(invoice.discount)}`);
  console.log(`কর (${taxPercent}%): ${formatTaka(invoice.tax)}`);
  console.log(`সর্বমোট: ${formatTaka(invoice.total)}`);
}

const items = [
  { name: 'Notebook', pricePaisa: 12050, quantity: 3 },
  { name: 'Pen', pricePaisa: 1500, quantity: 10 },
  { name: 'School Bag', pricePaisa: 95000, quantity: 1 },
];

printInvoice(items, 10, 5);
console.log(formatTaka(calculateInvoice(items).total));
```

Output

```
Notebook x 3: 361.50 টাকা
Pen x 10: 150.00 টাকা
School Bag x 1: 950.00 টাকা
উপমোট: 1461.50 টাকা
ছাড় (10%): 146.15 টাকা
কর (5%): 65.77 টাকা
সর্বমোট: 1381.12 টাকা
1461.50 টাকা
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Function-এর স্তর।** `printInvoice` Call করে `calculateInvoice`-কে, সেটি Call করে `lineTotal` ও `percentOf`-কে। প্রতিটি স্তর নিচের স্তরের নাম ব্যবহার করে, বিস্তারিত না জেনেই। ওপরের Function পড়লে কী হচ্ছে বোঝা যায়; নিচেরগুলো পড়লে কীভাবে।
- **Side Effect এক জায়গায়।** পুরো File-এ `console.log()` আছে শুধু `printInvoice`-এ এবং শেষ লাইনে। বাকি চারটি Function Pure, তাই প্রতিটিকে আলাদাভাবে পরীক্ষা করা যায়।
- **Scope।** `subtotal`, `discount`, `taxable`, `tax` —সবই `calculateInvoice`-এর Function Scope-এ। `printInvoice` এগুলো সরাসরি দেখতে পায় না; পায় শুধু ফেরত আসা Object-টি। `taxable` ফেরত দেওয়া হয়নি, তাই বাইরে থেকে সেটি পাওয়ার কোনো উপায় নেই।
- **Lexical Scope।** `formatTaka` নিজের ভেতরে `PAISA_PER_TAKA` পায় না; এক ধাপ বাইরে Global Scope-এ পায়। এটি একটি `const`, তাই এই নির্ভরতা নিরাপদ।
- **Hoisting।** `formatTaka` ও `lineTotal` Arrow Function, তাই ব্যবহারের আগে লেখা হয়েছে। `printInvoice`-এর ভেতরে এগুলোর ব্যবহার সমস্যা নয়, কারণ `printInvoice` Call হয় File-এর একেবারে শেষে, যখন সবকিছু তৈরি।
- **হিসাব।** উপমোট ১,৪৬,১৫০ পয়সা। ১০ শতাংশ ছাড় ১৪,৬১৫। বাকি ১,৩১,৫৩৫-এর ৫ শতাংশ ৬,৫৭৬.৭৫, যা নিকটতম পয়সায় ৬,৫৭৭। সর্বমোট ১,৩৮,১১২ পয়সা, অর্থাৎ ১৩৮১.১২ টাকা।

শেষ লাইনে একই `calculateInvoice` শুধু `items` দিয়ে Call করা হয়েছে; Default Parameter-এর কারণে ছাড় ও কর শূন্য, এবং ফল উপমোটের সমান।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node invoice.js
```

Call	যা পাওয়ার কথা
`calculateInvoice(items, 10, 5)`	উপমোট 146150, ছাড় 14615, কর 6577, সর্বমোট 138112 (পয়সায়)
`calculateInvoice(items)`	ছাড় 0, কর 0, সর্বমোট 146150
`calculateInvoice([])`	চারটি অঙ্কই 0; কোনো Error নয়
`percentOf(131535, 5)`	6577 (6576.75 নিকটতম পয়সায়)
`formatTaka(12050)`	120.50 টাকা
`formatTaka(5)`	0.05 টাকা

Companion Source Code-এ এই Project-এর ১৪টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`ReferenceError: Cannot access 'formatTaka' before initialization`	Arrow Function-টি ব্যবহারের পরে লেখা	সহায়ক Function-গুলো File-এর ওপরে রাখুন
সর্বমোট `NaN`	`discountPercent` বা `taxPercent`-এর Default নেই এবং Argument দেওয়া হয়নি	`= 0` যোগ করুন
`invoice.subtotal` হলো `undefined`	`calculateInvoice`-এ `return` নেই, বা Object-এর নামে বানান ভুল	`return { ... }` এবং নামগুলো মিলিয়ে দেখুন
অঙ্ক একশ গুণ বড় বা ছোট	কোথাও টাকা আর পয়সা মিশে গেছে	সব হিসাব পয়সায়; টাকায় শুধু `formatTaka`
কর বেশি আসছে	কর `subtotal`-এর ওপর হিসাব হচ্ছে	`taxable`-এর ওপর হিসাব করুন
`formatTaka` দশমিকের পরে অনেক ঘর দেখায়	`toFixed(2)` নেই	`(paisa / 100).toFixed(2)`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। পণ্যের তথ্য যাচাই করা হয় না: দাম ও পরিমাণ সঠিক পূর্ণসংখ্যা বলে ধরে নেওয়া হয়েছে, এবং শতাংশ ০ থেকে ১০০-এর মধ্যে আছে কি না তা দেখা হয় না (অধ্যায় ৩ ও ৪-এর কৌশলে তা যোগ করা যায়; অনুশীলনী ১২ দেখুন)। করের হার উদাহরণমূলক। Invoice সংরক্ষণ বা ছাপানোর ব্যবস্থা নেই।

৫.১৬ অনুশীলনী

প্রাথমিক স্তর

১. `square` নামে একটি Function Declaration লিখুন, যা একটি সংখ্যার বর্গ ফেরত দেয়। একই কাজ একটি Arrow Function দিয়েও লিখুন।

২. না চালিয়ে বলুন নিচের Code কী দেখাবে, এবং কেন।

predict-return.js · JavaScript চলবে: Browser Console ও Node.js

```
function sayHello(name) {
  console.log(`Hello, ${name}`);
}

const result = sayHello('Sara');
console.log(result);
```

৩. `fullName` নামে একটি Function লিখুন, যা নামের প্রথম ও শেষ অংশ নিয়ে মাঝখানে একটি ফাঁকা জায়গা দিয়ে জোড়া লাগিয়ে ফেরত দেয়।

৪. `power` নামে একটি Function লিখুন, যা একটি সংখ্যা ও ঘাত নেয়; ঘাত না দিলে `2` ধরা হবে।

৫. নিচের Code-এ একটি Scope-সংক্রান্ত ভুল আছে। ভুলটি ঠিক করুন, যাতে `450` দেখায়।

scope-bug.js · JavaScript চলবে: Node.js

```
function calculate(amount) {
  const discounted = amount - 50;
}

calculate(500);
console.log(discounted);
```

মধ্যম স্তর

৬. `isLeapYear` নামে একটি Arrow Function লিখুন, যা বছর অধিবর্ষ হলে `true` দেয়। নিয়ম: ৪ দিয়ে বিভাজ্য এবং ১০০ দিয়ে নয়; অথবা ৪০০ দিয়ে বিভাজ্য।

৭. `average` নামে একটি Function লিখুন, যা সংখ্যার একটি Array নিয়ে গড় ফেরত দেয়; ফাঁকা Array হলে `0`।

৮. না চালিয়ে বলুন নিচের Code কী দেখাবে। Lexical Scope দিয়ে উত্তরটি ব্যাখ্যা করুন।

predict-scope.js · JavaScript চলবে: Browser Console ও Node.js

```
const label = 'বাইরের';

function show() {
  return label;
}

function run() {
  const label = 'ভেতরের';
  return show();
}

console.log(run());
```

৯. `applyTwice` নামে একটি Function লিখুন, যা একটি সংখ্যা ও একটি Function নেয় এবং Function-টি সংখ্যার ওপর পরপর দুবার প্রয়োগ করে।

চ্যালেঞ্জ স্তর

১০. **Delivery যোগ:** `totalWithDelivery` Function লিখুন, যা Invoice-এর সর্বমোট (পয়সায়) এবং Delivery-র খরচ নেয়; খরচ না দিলে ৬০০০ পয়সা ধরা হবে; সর্বমোট ৩,০০,০০০ পয়সা বা বেশি হলে Delivery ফ্রি।

১১. **নিয়ম পাঠানো:** `priceAfter` Function লিখুন, যা একটি অঙ্ক এবং ছাড়ের নিয়ম (একটি Function) নেয়। তিনটি নিয়ম তৈরি করুন: কোনো ছাড় নয়, ১৫ শতাংশ, এবং সরাসরি ২০০০ পয়সা।

১২. **হার যাচাই:** `isValidPercent` Function লিখুন, যা মানটি ০ থেকে ১০০-এর মধ্যে সংখ্যা হলে `true` দেয়। তারপর `safePercentOf` লিখুন, যা হার সঠিক না হলে `null` ফেরত দেয়।

৫.১৭ অনুশীলনীর সমাধান

সমাধান ১

square.js · JavaScript চলবে: Browser Console ও Node.js

```
function square(number) {
  return number * number;
}

const squareArrow = (number) => number * number;

console.log(square(7));
console.log(squareArrow(7));
```

Output

```
49
49
```

সমাধান ২ — প্রথমে `Hello, Sara`, তারপর `undefined`। `sayHello` বার্তাটি দেখায়, কিন্তু কিছু `return` করে না; তাই `result`-এর মান `undefined`।

সমাধান ৩

full-name.js · JavaScript — চলবে: Browser Console ও Node.js

```
function fullName(first, last) {
  return `${first} ${last}`;
}

console.log(fullName('Sadia', 'Rahman'));
```

Output

```
Sadia Rahman
```

সমাধান ৪

power.js · JavaScript — চলবে: Browser Console ও Node.js

```
function power(base, exponent = 2) {
  return base ** exponent;
}

console.log(power(5));
console.log(power(2, 10));
```

Output

```
25
1024
```

সমাধান ৫ — `discounted` Function-এর ভেতরে তৈরি, তাই বাইরে থেকে পাওয়া যায় না। মানটি `return` করে বাইরে একটি Variable-এ নিতে হবে।

scope-fixed.js · JavaScript — চলবে: Browser Console ও Node.js

```
function calculate(amount) {
  return amount - 50;
}

const discounted = calculate(500);
console.log(discounted);
```

Output

```
450
```

সমাধান ৬

leap-year.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isLeapYear = (year) =>
  (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0;

console.log(isLeapYear(2024));
console.log(isLeapYear(1900));
console.log(isLeapYear(2000));
console.log(isLeapYear(2025));
```

Output

```
true
false
true
false
```

`&&` ও `||` একসঙ্গে থাকায় বন্ধনী দেওয়া হয়েছে, যদিও `&&` এমনিতেই আগে হিসাব হয়।

সমাধান ৭

average.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function average(numbers) {
  if (numbers.length === 0) {
    return 0;
  }

  let sum = 0;

  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
  }

  return sum / numbers.length;
}

console.log(average([80, 90, 70]));
console.log(average([]));
```

Output

```
80
0
```

সমাধান ৮ — বাইরের। `show` লেখা হয়েছে Global Scope-এ, তাই সে `label` খোঁজে নিজের ভেতরে এবং তারপর Global-এ। `run`-এর ভেতরের `label` তার Scope Chain-এ নেই, যদিও Call করা হয়েছে সেখান থেকেই।

সমাধান ৯

apply-twice.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function applyTwice(value, operation) {
  return operation(operation(value));
}

const addTen = (number) => number + 10;
const double = (number) => number * 2;

console.log(applyTwice(5, addTen));
console.log(applyTwice(5, double));
```

Output

```
25
20
```

সমাধান ১০

delivery.js · JavaScript — চলবে: Browser Console ও Node.js

```
const FREE_DELIVERY_FROM = 300000;

function totalWithDelivery(total, deliveryFee = 6000) {
  return total >= FREE_DELIVERY_FROM ? total : total + deliveryFee;
}

console.log(totalWithDelivery(138112));
console.log(totalWithDelivery(138112, 12000));
console.log(totalWithDelivery(300000));
```

Output

```
144112
150112
300000
```

সমাধান ১১

price-after.js · JavaScript — চলবে: Browser Console ও Node.js

```
const noDiscount = (amount) => 0;
const fifteenPercent = (amount) => Math.round(amount * 15 / 100);
const flat2000 = (amount) => 2000;

function priceAfter(amount, discountRule) {
  return amount - discountRule(amount);
}

console.log(priceAfter(50000, noDiscount));
console.log(priceAfter(50000, fifteenPercent));
console.log(priceAfter(50000, flat2000));
```

Output

```
50000
42500
48000
```

সমাধান ১২

safe-percent.js · JavaScript — চলবে: Browser Console ও Node.js

```
const isValidPercent = (value) =>
  typeof value === 'number' && value >= 0 && value <= 100;

function safePercentOf(amount, percent) {
  if (!isValidPercent(percent)) {
    return null;
  }

  return Math.round(amount * percent / 100);
}

console.log(safePercentOf(1000, 10));
console.log(safePercentOf(1000, 150));
console.log(safePercentOf(1000, '10'));
```

Output

```
100
null
null
```

`NaN`-এর `typeof` হলো `number`, কিন্তু `NaN >= 0` মিথ্যা, তাই `NaN`-ও বাতিল হয়।

৫.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `function add(a, b) { return a + b; }`-এ `a` এবং `b` কী?

ক) Argument

খ) Parameter

গ) Return Value

ঘ) Global Variable

প্রশ্ন ২. যে Function-এ `return` নেই, সেটি কী ফেরত দেয়?

ক) `null`

খ) `0`

গ) `undefined`

ঘ) একটি Error

প্রশ্ন ৩. `function greet(name = 'অতিথি')`-কে `greet(null)` দিয়ে Call করলে `name`-এর মান কী হয়?

ক) `null`

খ) `'অতিথি'`

গ) `undefined`

ঘ) ফাঁকা String

প্রশ্ন ৪. `const double = (n) => { n * 2; };` লিখে `double(5)` Call করলে ফল কী?

ক) `10`

খ) `5`

গ) একটি Error

ঘ) `undefined`

প্রশ্ন ৫. `function add(a, b) { return a + b; }` থাকলে `add(2)`-এর ফল কী?

ক) `2`

খ) `NaN`

গ) একটি Error

ঘ) `undefined`

প্রশ্ন ৬. একটি Function-এর ভেতরে `const` দিয়ে তৈরি Variable কোথা থেকে পাওয়া যায়?

ক) শুধু সেই Function-এর ভেতর থেকে

খ) File-এর সব জায়গা থেকে

গ) যে Function তাকে Call করেছে সেখান থেকেও

ঘ) শুধু Global Scope থেকে

প্রশ্ন ৭. Lexical Scope অনুযায়ী, একটি Function কোন Variable দেখতে পাবে তা কিসের ওপর নির্ভর করে?

ক) Function-টি কোথা থেকে Call করা হয়েছে

খ) Function-টি কতবার Call করা হয়েছে

গ) Function-টি Code-এর কোথায় লেখা হয়েছে

ঘ) Function-টির নাম

প্রশ্ন ৮. কোনটি তার লেখার লাইনের আগে Call করা যায়?

ক) `const f = () => 1;`

খ) `const f = function () { return 1; };`

গ) `let f = () => 1;`

ঘ) `function f() { return 1; }`

প্রশ্ন ৯. Pure Function-এর বৈশিষ্ট্য কোনটি?

ক) একই Argument-এ একই ফল দেয় এবং বাইরের কিছু বদলায় না

খ) সব সময় `console.log()` ব্যবহার করে

গ) কোনো Parameter নেয় না

ঘ) Global Variable বদলায়

প্রশ্ন ১০. `finalPrice(1000, tenPercent)`-এ `tenPercent` বন্ধনী ছাড়া লেখা হয়েছে। এর অর্থ কী?

ক) এটি একটি ভুল; বন্ধনী দিতে হবে

খ) Function-টি নিজেই পাঠানো হচ্ছে, তার ফলাফল নয়

গ) Function-টি সঙ্গে সঙ্গে চলে যায়

ঘ) `tenPercent` একটি String

৫.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Function তৈরির সময় লেখা নামগুলো Parameter। Call-এর সময় দেওয়া মানগুলো Argument।
২	গ	`return` না থাকলে, বা মান ছাড়া `return` থাকলে, ফল `undefined`।
৩	ক	Default শুধু `undefined`-এর জন্য বসে। `null` একটি ইচ্ছাকৃত মান।
৪	ঘ	`{ }` থাকলে `return` নিজে লিখতে হয়। এখানে হিসাব হয়, কিন্তু ফেরত যায় না।
৫	খ	`b` হয় `undefined`; `2 + undefined` এর ফল `NaN`। কোনো Error হয় না।
৬	ক	এটি Function Scope। বাইরে থেকে পেতে হলে মানটি `return` করতে হয়।
৭	গ	Scope ঠিক হয় লেখার জায়গা থেকে; Call-এর জায়গা কোনো প্রভাব ফেলে না।
৮	ঘ	শুধু Function Declaration পুরোপুরি Hoist হয়। বাকিগুলো `let` বা `const`-এর নিয়ম মানে।
৯	ক	Pure Function শুধু নিজের Argument-এর ওপর নির্ভর করে, তাই বোঝা ও পরীক্ষা করা সহজ।
১০	খ	বন্ধনী দিলে Function তখনই চলত। বন্ধনী ছাড়া নামটি Function-টিকেই বোঝায়।

৫.২০ অধ্যায়ের সারসংক্ষেপ

- Function হলো নামযুক্ত, পুনর্ব্যবহারযোগ্য Code; নাম দেয়, পুনরাবৃত্তি কমায় এবং পরিবর্তন এক জায়গায় আনে।
- Parameter হলো Function-এর নিজস্ব নাম; Argument হলো Call-এর সময় দেওয়া মান। সংখ্যা না মিললে Error হয় না।
- `return` মান ফেরত দেয় এবং Function থামায়। `console.log()` শুধু দেখায়।
- Default Parameter শুধু `undefined`-এর ক্ষেত্রে বসে।
- Function একটি মান: Variable-এ রাখা যায় (Function Expression), সংক্ষেপে লেখা যায় (Arrow Function), অন্য Function-কে পাঠানো যায় (Callback)।
- Arrow Function-এ `{ }` না থাকলে মান নিজে ফেরত যায়; থাকলে `return` লিখতে হয়।
- Scope তিন স্তরের: Global, Function, Block। ভেতর থেকে বাইরে দেখা যায়, বাইরে থেকে ভেতরে নয়।
- Lexical Scope: Function কী দেখবে তা ঠিক হয় কোথায় লেখা তা থেকে।
- Hoisting: Function Declaration আগে Call করা যায়; `var` তখন `undefined`; `let` ও `const` Error দেয়।
- ভালো Function ছোট, একটি কাজ করে, এবং যতটা সম্ভব Pure।

পরের অধ্যায়ে Array এবং Data প্রক্রিয়াকরণ: তালিকা তৈরি, বদলানো, এবং `map`, `filter`, `reduce` দিয়ে রূপান্তর—যেখানে এই অধ্যায়ের Callback সরাসরি কাজে লাগবে।

৫.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Function Declaration	`function name() { }` দিয়ে তৈরি Function।
Function Expression	Variable-এ রাখা Function।
Arrow Function	`=>` দিয়ে লেখা সংক্ষিপ্ত Function।
Call	Function চালানো।
Parameter / Argument	Function-এর গ্রহণের নাম / Call-এ দেওয়া মান।
Return Value	Function যে মান ফেরত দেয়।
Default Parameter	Argument না দিলে যে মান ব্যবহৃত হয়।
Scope	যে অংশ থেকে একটি Variable ব্যবহার করা যায়।
Scope Chain	নাম খোঁজার পথ: ভেতর থেকে বাইরের দিকে।
Shadowing	ভেতরের একই নামের Variable বাইরেরটিকে ঢেকে দেওয়া।
Lexical Scope	Scope ঠিক হয় Code কোথায় লেখা তা থেকে।

পরিভাষা	অর্থ
Hoisting	কিছু Declaration-এর, লেখার লাইনের আগেই পরিচিত থাকা।
Pure Function	একই Argument-এ একই ফল; বাইরের কিছু বদলায় না।
Side Effect	বাইরের অবস্থা বদলানো বা বাইরে কিছু দেখানো।
Callback	অন্য Function-কে Argument হিসেবে পাঠানো Function।

৫.২২ আরও পড়ুন

- **MDN — Functions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions` — Function-এর পূর্ণ নির্দেশিকা।
- **MDN — Arrow function expressions:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions` — Arrow Function-এর নিয়ম ও সীমা।
- **MDN — Default parameters:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters` — Default Parameter।
- **MDN — Scope:** `https://developer.mozilla.org/en-US/docs/Glossary/Scope` — Scope-এর সংজ্ঞা।
- **MDN — Hoisting:** `https://developer.mozilla.org/en-US/docs/Glossary/Hoisting` — Hoisting-এর ব্যাখ্যা।